

Nonlinear-Manifold ROM (NM-ROM)

From Training to Poisson and Burgers Prediction

A numerical walkthrough of the separable-decoder architecture

Purpose of this document. Part I follows Poisson from training to a final prediction with small numbers. Part II explains the same architecture as a general nonlinear-manifold ROM. Part III shows exactly what changes for time-dependent Burgers. The Poisson project dimensions in Section 2 and the Burgers dimensions in Section 12.1 come from the implemented configurations. The small arithmetic examples are teaching numbers, not measured results.

Contents

1	The problem and the model	3
2	The real project dimensions	3
3	Offline training	4
4	One-time Poisson precomputation	6
5	Online solve for a new source	8
6	Convergence and final prediction	9
7	What is fixed, and what changes?	10
8	Why the implementation looks slightly shorter	11
9	The Poisson method in twelve lines	11
10	How this becomes a general NM-ROM	12
11	The general offline and online recipe	12
12	Burgers: what changes from Poisson	13
13	Burgers one-time precomputation	14
14	The Burgers residual, term by term	16
15	One complete Burgers rollout	17
16	The important upwind sign caveat	17

17 Poisson and Burgers side by side	18
18 The full NM-ROM pipeline in fifteen lines	18
Plain-language glossary	20

1 The problem and the model

We want to solve the Poisson equation

$$-\Delta u = f, \quad u = 0 \text{ on the boundary.} \quad (1)$$

Here f is a new source field and u is the solution we want to predict.

The learned decoder has two separate neural networks:

$$u(\mathbf{x}; \mathbf{z}) = s b_c(\mathbf{x}) \mathbf{g}(\mathbf{x})^\top \mathbf{h}(\mathbf{z}). \quad (2)$$

The two networks have different jobs:

- $\mathbf{g}(\mathbf{x})$ produces R spatial-pattern values at coordinate $\mathbf{x}$. Here R means the number of learned spatial patterns.
- $\mathbf{h}(\mathbf{z})$ produces R coefficients, one coefficient for each of those R learned spatial patterns.
- $b_c(\mathbf{x})$ is zero on the boundary, so every decoded field satisfies the boundary condition automatically.

On all interior grid points at once, the decoder becomes

$$\mathbf{u}(\mathbf{z}) = \mathbf{G} \mathbf{h}(\mathbf{z}), \quad \mathbf{G} \in \mathbb{R}^{P \times R}, \quad (3)$$

Here P is the number of interior grid values in the solution vector, and R is the number of learned spatial patterns. Thus $\mathbf{G}$ has one row per grid value and one column per learned pattern. In the implementation, row i of $\mathbf{G}$ is

$$G_{i,:} = s b_c(\mathbf{x}_i) \mathbf{g}(\mathbf{x}_i)^\top, \quad (4)$$

so the fixed training-data scale s and the hard boundary mask are already inside the cached bank.

The new forcing f is *not* fed into $\mathbf{h}$. For a new problem, f enters the residual. A numerical solver changes $\mathbf{z}$ until the decoded field satisfies the projected Poisson equation.

Three sizes to remember throughout the document. P counts physical-grid values in $\mathbf{u}$; K counts the latent variables in $\mathbf{z}$ that the online solver changes; and R counts both the learned spatial patterns in $\mathbf{G}$ and their coefficients in $\mathbf{h}(\mathbf{z})$. The same reminder appears in every page footer.

2 The real project dimensions

For the $N = 256$ Poisson quadrature-free run, the full mesh has 256×256 points. Removing the outer boundary leaves

$$P = (256 - 2)^2 = 254^2 = 64,516 \quad (5)$$

interior points. In other words, P is the length of a full interior solution vector, not the number of latent variables or learned patterns.

The main dimensions are:

Object	Shape	Meaning
$\mathbf{z}$	$16 = K$	latent variables changed by the solver
$\mathbf{h}(\mathbf{z})$	$64 = R$	one coefficient per learned spatial pattern
$\mathbf{G}$	$64,516 \times 64 = P \times R$	grid values by learned patterns
$\Phi^\top$	$64 \times 64,516$	64 sine test functions
$\mathbf{B}$	64×64	projected spatial bank
$\mathbf{A}$	64×64	projected Poisson operator
$\mathbf{r}(\mathbf{z})$	64	residual seen by the latent solver
$\mathbf{J}_r$	64×16	residual Jacobian with respect to $\mathbf{z}$

The training configuration used 512 training solutions and 100,000 optimizer steps, with

$$K = 16, \quad R = 64, \quad M = 64. \quad (6)$$

That means 16 latent variables are adjusted online ($K = 16$), the decoder uses 64 learned spatial patterns and 64 matching coefficients ($R = 64$), and the residual retains 64 test equations ($M = 64$).

Notation warning. The blackboard-bold symbol $\mathbb{R}$ in an expression such as $\mathbb{R}^{16}$ means “vectors of real numbers.” It is not the model-size symbol R . For example, $\mathbb{R}^{16}$ below is a real-valued vector with 16 entries, while $R = 64$ counts learned patterns.

The spatial path begins with 64 learned frequency vectors, expanded into 64 sine values and 64 cosine values. Its architecture is

$$\mathbb{R}^2 \longrightarrow 128 \text{ Fourier features} \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{64}, \quad (7)$$

with SiLU activations after the two hidden dense layers and a linear final layer. Its output is multiplied by the fixed training-data RMS scale and the hard boundary mask when the bank is formed. The latent path is

$$\mathbb{R}^{16} \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{64}, \quad (8)$$

again with SiLU hidden activations and a linear final layer. A separate linear map from $\mathbb{R}^{16}$ to $\mathbb{R}^{64}$ is added as a skip connection. In this path, the 16 input entries are the $K = 16$ latent variables, and the 64 output entries are the $R = 64$ learned-pattern coefficients.

3 Offline training

3.1 Step 1: generate solution snapshots

The full-order Poisson solver produces training pairs

$$(\mathbf{f}_s, \mathbf{u}_s), \quad s = 1, \dots, 512. \quad (9)$$

Each $\mathbf{u}_s$ contains $P = 64,516$ interior solution values in the $N = 256$ run; here P again means the number of physical-grid values stored in one solution snapshot.

3.2 Step 2: assign a trainable code to every snapshot

Because this is an auto-decoder, there is no encoder. Every training snapshot gets its own trainable latent vector:

$$\mathbf{z}_s \in \mathbb{R}^{16}. \quad (10)$$

The 16 entries of this vector are the $K = 16$ latent variables assigned to that snapshot. The 512 latent codes and the weights of both neural networks are optimized together.

3.3 Step 3: reconstruct every training solution

At a training step, evaluate the spatial bank below. It has P rows, one for each interior grid value, and R columns, one for each learned spatial pattern:

$$\mathbf{G}_{\theta_g} = \begin{bmatrix} s b_c(\mathbf{x}_1) \mathbf{g}_{\theta_g}(\mathbf{x}_1)^\top \\ \vdots \\ s b_c(\mathbf{x}_P) \mathbf{g}_{\theta_g}(\mathbf{x}_P)^\top \end{bmatrix} \in \mathbb{R}^{P \times R} \quad (11)$$

The coefficient matrix below has 512 rows, one for each training snapshot, and R columns, one coefficient for each learned spatial pattern:

$$\mathbf{H}_{\theta_h} = \begin{bmatrix} \mathbf{h}_{\theta_h}(\mathbf{z}_1)^\top \\ \vdots \\ \mathbf{h}_{\theta_h}(\mathbf{z}_{512})^\top \end{bmatrix} \in \mathbb{R}^{512 \times R}. \quad (12)$$

All training reconstructions are produced together by the following matrix. It has 512 rows, one reconstructed snapshot per row, and P columns, one for each interior grid value:

$$\hat{\mathbf{U}} = \mathbf{H}_{\theta_h} \mathbf{G}_{\theta_g}^\top \in \mathbb{R}^{512 \times P}. \quad (13)$$

The main loss is the global relative mean-square reconstruction error,

$$\mathcal{L}_{\text{recon}} = \frac{\|\hat{\mathbf{U}} - \mathbf{U}\|_F^2}{\|\mathbf{U}\|_F^2}, \quad (14)$$

plus a small feature-Gram regularizer that discourages badly conditioned or redundant columns in $\mathbf{G}$.

Adam updates

$$\theta_g, \quad \theta_h, \quad \mathbf{z}_1, \dots, \mathbf{z}_{512}. \quad (15)$$

3.4 A tiny training calculation

For the worked example, use only four interior points, two latent variables, and three spatial features:

$$P = 4, \quad K = 2, \quad R = 3. \quad (16)$$

Thus $P = 4$ means four physical-grid values in each solution, $K = 2$ means two latent variables for the solver to adjust, and $R = 3$ means three learned spatial patterns and three matching coefficients.

Suppose one training target is

$$\mathbf{u}_{\text{train}} = [1 \quad 0 \quad 1 \quad 2]^\top. \quad (17)$$

An early prediction might be

$$\hat{\mathbf{u}} = [0.3 \quad -0.1 \quad 0.2 \quad 0.4]^\top. \quad (18)$$

Its pointwise error and plain mean-square error are

$$\hat{\mathbf{u}} - \mathbf{u}_{\text{train}} = [-0.7 \quad -0.1 \quad -0.8 \quad -1.6]^\top, \quad (19)$$

$$\text{MSE} = \frac{0.49 + 0.01 + 0.64 + 2.56}{4} = 0.925. \quad (20)$$

This plain MSE is only a small arithmetic illustration; the production loss uses the relative normalization written above.

After many updates, suppose training has produced the bank

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix}. \quad (21)$$

For the training snapshot above, suppose the trained head gives

$$\mathbf{h}(\mathbf{z}_{\text{train}}) = [1 \ 0 \ 0]^\top. \quad (22)$$

Then

$$\mathbf{G}\mathbf{h}(\mathbf{z}_{\text{train}}) = [1 \ 0 \ 1 \ 2]^\top, \quad (23)$$

so this teaching snapshot is reconstructed exactly.

3.5 Step 4: freeze the learned model

After training, the spatial network, Fourier frequencies, latent network, and linear skip are frozen. The training codes are no longer optimized. They may still be useful for constructing an initial guess for a new solve.

4 One-time Poisson precomputation

4.1 Step 5: evaluate the frozen spatial bank

Evaluate the frozen spatial network at every interior point, multiply by the fixed output scale and hard boundary mask, and build a matrix with P rows (one per interior grid value) and R columns (one per learned spatial pattern):

$$\mathbf{G} \in \mathbb{R}^{P \times R}. \quad (24)$$

This is the large spatial calculation. It is not repeated during each latent solver iteration.

4.2 Step 6: construct the sine test functions

Let

$$\Phi = [\phi_1 \ \cdots \ \phi_M] \in \mathbb{R}^{P \times M} \quad (25)$$

contain the selected discrete sine modes. The P rows place every test function on all P interior grid values; the M columns are the M retained test functions. They are eigenvectors of the discrete negative Laplacian $\mathbf{L}$:

$$\mathbf{L}\phi_m = \lambda_m\phi_m. \quad (26)$$

For a general fixed linear operator, the projected matrix would be built as

$$\mathbf{A} = \Phi^\top \mathbf{L} \mathbf{G}. \quad (27)$$

For Poisson sine modes, symmetry and the eigenvector identity give the shortcut

$$\mathbf{A} = \mathbf{\Lambda} \mathbf{B}, \quad \mathbf{B} = \Phi^\top \mathbf{G}, \quad (28)$$

where

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_M). \quad (29)$$

4.3 Step 7: compute every small matrix in the toy example

Take a 4×4 full grid on $[0, 1]^2$. It has a 2×2 interior, so $P = 4$ physical interior-grid values, and its grid spacing is $\Delta x = 1/3$. For readable arithmetic, manually select two test modes, $M = 2$. Their values are

$$\Phi^\top = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix}. \quad (30)$$

These are actual normalized two-dimensional discrete sine modes on this tiny grid. The production mode selector retains complete eigenvalue shells. On this unusually small grid, requesting two modes would therefore retain the second mode's tied partner as a third mode. The teaching example intentionally omits that tied partner to keep every displayed matrix two rows tall; each displayed vector remains a valid sine eigenmode. Their negative-Laplacian eigenvalues are

$$\lambda_1 = 18, \quad \lambda_2 = 36, \quad \Lambda = \begin{bmatrix} 18 & 0 \\ 0 & 36 \end{bmatrix}. \quad (31)$$

Projecting the bank in Equation (21) gives

$$\mathbf{B} = \Phi^\top \mathbf{G} \quad (32)$$

$$= \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix} \quad (33)$$

$$= \begin{bmatrix} 2 & 1.5 & 0.5 \\ 0 & -0.5 & 0.5 \end{bmatrix}. \quad (34)$$

For example, the upper-left entry is

$$B_{11} = 0.5(1) + 0.5(0) + 0.5(1) + 0.5(2) = 2. \quad (35)$$

The projected Poisson matrix is

$$\mathbf{A} = \Lambda \mathbf{B} \quad (36)$$

$$= \begin{bmatrix} 18 & 0 \\ 0 & 36 \end{bmatrix} \begin{bmatrix} 2 & 1.5 & 0.5 \\ 0 & -0.5 & 0.5 \end{bmatrix} \quad (37)$$

$$= \begin{bmatrix} 36 & 27 & 9 \\ 0 & -18 & 18 \end{bmatrix}. \quad (38)$$

For the project choice $\alpha = 1$, the residual weighting is the inverse eigenvalue scaling

$$\mathbf{W} = \Lambda^{-1} = \begin{bmatrix} 1/18 & 0 \\ 0 & 1/36 \end{bmatrix}. \quad (39)$$

At this point, $\mathbf{G}$, Φ , $\mathbf{B}$, $\mathbf{A}$, Λ , and $\mathbf{W}$ are mathematically fixed. The optimized code does not need to materialize all of them as separate dense matrices: it stores $\mathbf{B}$, the eigenvalue vector, and the already scaled forcing.

5 Online solve for a new source

5.1 Step 8: receive and project the new forcing

Suppose the new forcing on the four interior points is

$$\mathbf{f} = [54 \quad 9 \quad 9 \quad 18]^\top. \quad (40)$$

Project it onto the two test functions once:

$$\mathbf{f}_M = \Phi^\top \mathbf{f} \quad (41)$$

$$= \begin{bmatrix} 0.5(54) + 0.5(9) + 0.5(9) + 0.5(18) \\ 0.5(54) - 0.5(9) + 0.5(9) - 0.5(18) \end{bmatrix} \quad (42)$$

$$= \begin{bmatrix} 45 \\ 18 \end{bmatrix}. \quad (43)$$

This source projection may touch the full grid, but it is performed only once per new source, not once per latent iteration. If the forcing has a known affine expansion, the projections of its fixed components can also be precomputed.

Notation warning. In this document, $\mathbf{f}_M$ means the raw projection $\Phi^\top \mathbf{f}$. The implementation variable named $\mathbf{f_m}$ instead stores the scaled vector $\Lambda^{-1} \Phi^\top \mathbf{f}$.

5.2 Step 9: evaluate one candidate latent state

Suppose the latent solver proposes

$$\mathbf{z} = [0.4 \quad -0.7]^\top \quad (44)$$

and the frozen latent MLP returns

$$\mathbf{c} = \mathbf{h}(\mathbf{z}) = [2 \quad -1 \quad 0.5]^\top. \quad (45)$$

The projected left side of the PDE is

$$\mathbf{A}\mathbf{c} = \begin{bmatrix} 36 & 27 & 9 \\ 0 & -18 & 18 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 0.5 \end{bmatrix} \quad (46)$$

$$= \begin{bmatrix} 36(2) + 27(-1) + 9(0.5) \\ 0(2) - 18(-1) + 18(0.5) \end{bmatrix} \quad (47)$$

$$= \begin{bmatrix} 49.5 \\ 27 \end{bmatrix}. \quad (48)$$

The raw projected residual is

$$\mathbf{A}\mathbf{h}(\mathbf{z}) - \mathbf{f}_M = \begin{bmatrix} 49.5 \\ 27 \end{bmatrix} - \begin{bmatrix} 45 \\ 18 \end{bmatrix} = \begin{bmatrix} 4.5 \\ 9 \end{bmatrix}. \quad (49)$$

After applying the weights,

$$\mathbf{r}(\mathbf{z}) = \mathbf{W}(\mathbf{A}\mathbf{h}(\mathbf{z}) - \mathbf{f}_M) \quad (50)$$

$$= \begin{bmatrix} 4.5/18 \\ 9/36 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.25 \end{bmatrix}. \quad (51)$$

The candidate is therefore not yet a solution.

5.3 Step 10: compute a latent update

Automatic differentiation is needed only through the small head. Suppose its local Jacobian is

$$\mathbf{J}_h = \frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & -0.5 \end{bmatrix} \in \mathbb{R}^{3 \times 2}. \quad (52)$$

Its three rows correspond to the $R = 3$ learned-pattern coefficients, while its two columns correspond to the $K = 2$ latent variables being adjusted. Then the residual Jacobian is

$$\mathbf{J}_r = \mathbf{W} \mathbf{A} \mathbf{J}_h \quad (53)$$

$$= \begin{bmatrix} 2.25 & 1.25 \\ 0.25 & -0.75 \end{bmatrix}. \quad (54)$$

For illustration, an undamped local Newton step solves

$$\mathbf{J}_r \delta \mathbf{z} = -\mathbf{r}, \quad (55)$$

which here gives

$$\delta \mathbf{z} = \begin{bmatrix} -0.25 \\ 0.25 \end{bmatrix}. \quad (56)$$

The production method uses Levenberg–Marquardt damping and a trust-region limit, so it may take a smaller step. It repeats

$$\mathbf{z} \longrightarrow \mathbf{h}(\mathbf{z}) \longrightarrow \mathbf{r}(\mathbf{z}) \longrightarrow \delta \mathbf{z} \quad (57)$$

until the residual or progress stopping condition is satisfied.

No spatial network, full-grid Laplacian, or $\Phi^\top \mathbf{G}$ product is evaluated inside this loop.

6 Convergence and final prediction

6.1 Step 11: reach a zero-residual coefficient vector

Suppose the final latent state $\mathbf{z}_\star$ produces

$$\mathbf{h}(\mathbf{z}_\star) = [1 \quad 0 \quad 1]^\top. \quad (58)$$

Then

$$\mathbf{A} \mathbf{h}(\mathbf{z}_\star) = \begin{bmatrix} 36(1) + 27(0) + 9(1) \\ 0(1) - 18(0) + 18(1) \end{bmatrix} \quad (59)$$

$$= \begin{bmatrix} 45 \\ 18 \end{bmatrix} = \mathbf{f}_M. \quad (60)$$

Therefore

$$\mathbf{r}(\mathbf{z}_\star) = \mathbf{0}. \quad (61)$$

6.2 Step 12: decode the full field once

The final interior prediction is

$$\mathbf{u}_\star = \mathbf{G}\mathbf{h}(\mathbf{z}_\star) \quad (62)$$

$$= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad (63)$$

$$= \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix}. \quad (64)$$

Restoring the zero boundary gives the 4×4 field

$$\mathbf{U}_\star = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (65)$$

For completeness, the full four-point negative-Laplacian matrix is

$$\mathbf{L} = 9 \begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix}. \quad (66)$$

Direct multiplication verifies that this deliberately constructed toy prediction satisfies the complete grid equation, not only the two retained tests:

$$\mathbf{L}\mathbf{u}_\star = \begin{bmatrix} 54 \\ 9 \\ 9 \\ 18 \end{bmatrix} = \mathbf{f}. \quad (67)$$

In a general reduced solve, a small retained test set enforces only the selected weak equations; this full equality occurs here because the teaching example was constructed to make it checkable end to end.

7 What is fixed, and what changes?

Fixed after training/precomputation	Changes for a new source or solver iterate
weights of $\mathbf{g}$ and $\mathbf{h}$	the source field $\mathbf{f}$
spatial bank $\mathbf{G}$	its projected vector $\mathbf{f}_M$ changes per source
test functions Φ	current latent vector $\mathbf{z}$
eigenvalues Λ	coefficients $\mathbf{h}(\mathbf{z})$
projected matrices $\mathbf{B}$ and $\mathbf{A}$	residual and residual Jacobian
weighting $\mathbf{W}$	final decoded field

The matrices must be rebuilt if the learned spatial bank, mesh, boundary conditions, test space, or PDE operator changes. A new right-hand side on the same problem requires a new $\mathbf{f}_M$, but not a new $\mathbf{A}$.

8 Why the implementation looks slightly shorter

The slide form is

$$\mathbf{r}(\mathbf{z}) = \mathbf{W} [\mathbf{\Lambda} \mathbf{B} \mathbf{h}(\mathbf{z}) - \mathbf{f}_M], \quad \mathbf{W} = \mathbf{\Lambda}^{-1}. \quad (68)$$

Since $\mathbf{W} \mathbf{\Lambda} = \mathbf{I}$, this is exactly

$$\mathbf{r}(\mathbf{z}) = \mathbf{B} \mathbf{h}(\mathbf{z}) - \mathbf{\Lambda}^{-1} \mathbf{f}_M. \quad (69)$$

The quadrature-free code uses this second form: it materializes $\mathbf{B} = \mathbf{\Phi}^\top \mathbf{G}$, not the conceptual dense matrix $\mathbf{A}$, and stores the forcing after inverse-eigenvalue scaling. The apparent disappearance of $\mathbf{W}$ and $\mathbf{\Lambda}$ is only an algebraic fold; it does not change the residual.

9 The Poisson method in twelve lines

1. Generate full-order Poisson training solutions.
2. Give each training solution a trainable latent code.
3. Train $\mathbf{g}$, $\mathbf{h}$, and the training codes to reconstruct the solutions.
4. Freeze both neural networks.
5. Evaluate $\mathbf{g}$ on the grid to construct $\mathbf{G}$.
6. Construct the sine test functions $\mathbf{\Phi}$ and eigenvalues $\mathbf{\Lambda}$.
7. Precompute $\mathbf{B} = \mathbf{\Phi}^\top \mathbf{G}$ and, conceptually, $\mathbf{A} = \mathbf{\Lambda} \mathbf{B}$.
8. For a new source, compute $\mathbf{f}_M = \mathbf{\Phi}^\top \mathbf{f}$ once.
9. Choose an initial latent vector $\mathbf{z}$.
10. Repeatedly evaluate the small head and residual, then update $\mathbf{z}$.
11. Stop at $\mathbf{z}_\star$ when the residual is sufficiently small.
12. Decode once: $\mathbf{u}_\star = \mathbf{G} \mathbf{h}(\mathbf{z}_\star)$.

The central computational saving: the full grid is used to build the spatial bank, to project a new source, and to return the final field. It is not revisited during every latent-solver iteration. Each iteration uses only the small latent MLP and precomputed dense matrices.

10 How this becomes a general NM-ROM

The decoder used above is not only a Poisson decoder. Its general form is still

$$\underbrace{\mathbf{z}}_{K \text{ latent variables}} \longrightarrow \underbrace{\mathbf{h}(\mathbf{z})}_{R \text{ coefficients}} \longrightarrow \underbrace{\mathbf{G}\mathbf{h}(\mathbf{z})}_{P \text{ physical-grid values}}. \quad (70)$$

Here P counts physical-grid values, K counts the latent variables changed by the online solver, and R counts the frozen learned spatial patterns in $\mathbf{G}$ and their matching coefficients in $\mathbf{h}(\mathbf{z})$.

Although $\mathbf{G}$ is fixed, the predicted solution is not fixed. The coefficient vector $\mathbf{h}(\mathbf{z})$ changes when $\mathbf{z}$ changes, so the same columns of $\mathbf{G}$ can make many different fields. A simple analogy is a fixed palette of R paint colors: keeping the palette fixed does not keep the painting fixed, because the mixture changes.

There are two nested geometric objects:

- All combinations $\mathbf{G}\mathbf{c}$, with any R -entry coefficient vector $\mathbf{c}$, form a linear space of dimension at most R .
- The neural head permits only coefficients of the form $\mathbf{c} = \mathbf{h}(\mathbf{z})$, where $\mathbf{z}$ has only K entries. This usually traces a curved, K -dimensional surface inside the larger R -dimensional coefficient space.

That curved surface is the learned *nonlinear manifold*. The spatial bank $\mathbf{G}$ can therefore be fixed while the allowed solutions depend nonlinearly on the K latent variables.

What “generalizes” means here. The trained model can solve new parameter values, source fields, initial conditions, or time states from the same problem family by finding a new latent vector. It does not automatically generalize to an unrelated PDE, boundary condition, or operator. A changed mesh requires the continuous spatial network to be reevaluated and all projected objects to be rebuilt; accuracy on that mesh must also be checked.

There is an important conditional statement for linear Poisson. If a forcing has a finite affine expansion

$$\mathbf{f}(\mu) = \sum_{a=1}^A \theta_a(\mu) \mathbf{f}_a, \quad (71)$$

then, for a fixed invertible Poisson matrix $\mathbf{L}$,

$$\mathbf{u}(\mu) = \sum_{a=1}^A \theta_a(\mu) \mathbf{L}^{-1} \mathbf{f}_a. \quad (72)$$

Those exact solutions lie in the linear span of the A fixed fields $\mathbf{L}^{-1} \mathbf{f}_a$, so a nonlinear head cannot improve representation accuracy; its possible value is reducing online cost. The implemented Poisson data use Gaussian sources, which are not affine in their center and width parameters. Therefore that theorem does not, by itself, prove that the implemented Gaussian solution family is a low-dimensional linear subspace. The Poisson residual precomputation remains exact because the operator is fixed and each new source enters only through its projected right-hand side.

11 The general offline and online recipe

Suppose a discretized PDE is written as

$$\mathbf{F}(\mathbf{u}; \mathbf{d}) = \mathbf{0}. \quad (73)$$

The vector $\mathbf{u}$ has P physical-grid values. The symbol $\mathbf{d}$ collects the data that define the particular problem, such as a forcing, viscosity, previous time state, or boundary data.

After auto-decoder training, freeze $\mathbf{G}$ and the head $\mathbf{h}$. Choose M smooth test functions and store them as columns of $\Phi \in \mathbb{R}^{P \times M}$. Here M is the number of weak test equations, whereas P is the number of physical-grid values. The online solver minimizes the weighted weak residual

$$\mathbf{r}(\mathbf{z}; \mathbf{d}) = \mathbf{W}\Phi^\top \mathbf{F}(\mathbf{G}\mathbf{h}(\mathbf{z}); \mathbf{d}). \quad (74)$$

This says: decode a candidate field, apply the discrete PDE, project the mismatch onto M smooth test functions, and adjust the K latent variables until that projected mismatch is small.

The amount that can be precomputed is determined by the algebraic degree of the PDE residual as a function of the state $\mathbf{u}$:

State dependence	Typical term	What is stored before the online solve
Degree 1: linear	diffusion, Poisson, Stokes	an $M \times R$ projected matrix
Degree 2: quadratic	Burgers advection, fluid convection	an $M \times R \times R$ tensor
Non-polynomial	$ u $, $1/\rho$, limiters	exact polynomial pieces, plus samples or another approximation for the rest

For example, a fixed linear term $\mathbf{L}\mathbf{u}$ becomes

$$\Phi^\top \mathbf{L}\mathbf{G}\mathbf{h}(\mathbf{z}). \quad (75)$$

Everything before $\mathbf{h}(\mathbf{z})$ is one precomputed $M \times R$ matrix. A quadratic term contains products of two entries of $\mathbf{h}(\mathbf{z})$, so its coefficients form a three-index tensor: one index for the M test equations and two indices for pairs of the R learned patterns.

12 Burgers: what changes from Poisson

The implemented one-dimensional model is viscous Burgers on the interval $[0, 1]$ with zero values at both walls. In continuous notation,

$$u_t + u u_x = \nu u_{xx}. \quad (76)$$

The viscosity ν controls diffusion. Small ν permits a sharper moving front; larger ν smooths it more strongly.

Backward Euler turns one time step into the nonlinear equation

$$\underbrace{\mathbf{u}^{n+1} - \mathbf{u}^n}_{\text{change over one step}} + \Delta t \left[\underbrace{\mathbf{N}(\mathbf{u}^{n+1})}_{\text{advection}} - \underbrace{\nu \Delta_h \mathbf{u}^{n+1}}_{\text{diffusion}} \right] = \mathbf{0}. \quad (77)$$

Here n labels the known old time, $n+1$ labels the unknown new time, $\Delta t = 0.005$ is the time-step size, ν is the viscosity, and Δ_h is the discrete second derivative. The implementation takes 50 steps. Therefore one initial field produces 51 saved states: the initial state plus 50 new states.

Poisson has one solve for one source. Burgers instead has a chain of solves. At every time step, the previous decoded state is known and the next K -entry latent vector must be found.

12.1 The real Burgers sizes

For a concrete $N = 256$ Burgers grid, two points are boundary points and

$$P = 256 - 2 = 254 \quad (78)$$

physical interior-grid values remain. The implemented model sizes are

$$K = 8, \quad R = 32, \quad M = 32. \quad (79)$$

Thus the online solver adjusts $K = 8$ latent variables; the frozen spatial bank contains $R = 32$ learned patterns and the head returns 32 matching coefficients; and the weak residual has $M = 32$ test equations.

Object	Shape at $N = 256$	Plain meaning
$\mathbf{z}$	$8 = K$	latent unknowns in one time-step solve
$\mathbf{h}(\mathbf{z})$	$32 = R$	coefficients of the learned patterns
$\mathbf{G}$	$254 \times 32 = P \times R$	learned patterns on the interior grid
Φ	$254 \times 32 = P \times M$	32 sine test functions on that grid
$\mathbf{B} = \Phi^\top \mathbf{G}$	$32 \times 32 = M \times R$	projected spatial bank
$\mathbf{Q}$	$32 \times 32 \times 32 = M \times R \times R$	projected pairwise advection interactions
$\mathbf{r}$	$32 = M$	weak residual for one candidate state
$\mathbf{J}_r$	$32 \times 8 = M \times K$	residual sensitivity to the latent variables

The training generator produces 512 trajectories. Each has 51 states, so 26,112 states are available; the configuration selects at most 8,192 of them for auto-decoder training. Every selected state receives its own $K = 8$ entry training code. The networks learn one P -by- R spatial bank and a nonlinear map from those eight latent variables to $R = 32$ coefficients, exactly as in the Poisson training description.

The Burgers spatial network starts from one coordinate x . Sixty-four learned frequencies produce 64 sine and 64 cosine values, giving 128 Fourier features. The path is

$$\mathbb{R}^1 \longrightarrow 128 \text{ Fourier features} \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{32}. \quad (80)$$

Its 32 outputs are the $R = 32$ raw learned-pattern values at x . The stored bank row is actually

$$G(x, :) = s 4x(1-x) \tilde{\mathbf{g}}(x)^\top, \quad (81)$$

where s is the fixed root-mean-square amplitude of the training data. The factor $4x(1-x)$ makes every pattern zero at both walls. The Burgers latent head is

$$\mathbb{R}^8 \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{32}, \quad (82)$$

plus a separate linear skip from its $K = 8$ latent inputs to its $R = 32$ coefficient outputs. Both hidden layers use the SiLU activation.

13 Burgers one-time precomputation

The $M = 32$ lowest discrete sine modes are used as test functions. They are eigenvectors of the negative discrete Laplacian, so

$$-\Delta_h \Phi = \Phi \Lambda, \quad (83)$$

where $\mathbf{\Lambda}$ contains M positive eigenvalues. First precompute

$$\mathbf{B} = \mathbf{\Phi}^\top \mathbf{G} \in \mathbb{R}^{M \times R}. \quad (84)$$

Here M counts test equations and R counts learned patterns. This single matrix handles both the state-change term and, with the eigenvalues, the linear diffusion term in Equation (77).

The advection term is quadratic. At interior grid node i , a decoded field value is

$$u_i = \sum_{k=1}^R G_{ik} h_k, \quad (85)$$

a backward difference gives

$$(D^- u)_i = \sum_{k=1}^R (D^- G)_{ik} h_k. \quad (86)$$

Multiplying the two expressions produces every pair of coefficients $h_j h_k$. Projecting that product onto test function m gives

$$T_{mjk} = \sum_{i=1}^P \Phi_{im} G_{ij} (D^- G)_{ik}. \quad (87)$$

The index m selects one of the M test equations. The indices j and k select an ordered pair of the R learned spatial patterns. Consequently $\mathbf{T}$ has shape $M \times R \times R$.

The raw tensor is not symmetric because one bank factor is differenced and the other is not. Store the symmetrized tensor

$$\mathbf{Q}_{mjk} = T_{mjk} + T_{mkj}. \quad (88)$$

For one R -entry coefficient vector $\mathbf{h}$, the complete projected advection vector is then

$$q_m(\mathbf{h}) = \frac{1}{2} \mathbf{h}^\top \mathbf{Q}_m \mathbf{h}, \quad m = 1, \dots, M. \quad (89)$$

There is one $R \times R$ matrix slice $\mathbf{Q}_m$ for each test equation. With $M = R = 32$, the production tensor contains $32^3 = 32,768$ numbers, which occupy 256 KiB in 64-bit floating point.

13.1 A two-coefficient tensor calculation

This tiny arithmetic example explains the contraction in Equation (89). Suppose one test equation has

$$\mathbf{Q}_1 = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}. \quad (90)$$

Here the teaching example has $R = 2$ learned-pattern coefficients. First,

$$\mathbf{Q}_1 \mathbf{h} = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 9 \end{bmatrix}. \quad (91)$$

Then

$$q_1(\mathbf{h}) = \frac{1}{2} \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 4 \\ 9 \end{bmatrix} = \frac{1}{2} (22) = 11. \quad (92)$$

That one number is the advection contribution to one test equation. The real model performs the same operation for all $M = 32$ test equations and all $R = 32$ coefficients at once.

14 The Burgers residual, term by term

Assume the old latent state $\mathbf{z}^n$ is known. Its projected physical state is computed once as

$$\mathbf{p}^n = \mathbf{B}\mathbf{h}(\mathbf{z}^n) = \Phi^\top \mathbf{u}^n. \quad (93)$$

The vector $\mathbf{p}^n$ has M entries, one per test equation. It does not have P entries because the known field has already been projected.

For a candidate new latent state $\mathbf{z}$, write

$$\mathbf{c} = \mathbf{h}(\mathbf{z}). \quad (94)$$

The vector $\mathbf{z}$ has $K = 8$ latent variables and the vector $\mathbf{c}$ has $R = 32$ learned-pattern coefficients. The exact small residual used by the tensor arm is

$$\mathbf{r}(\mathbf{z}) = \mathbf{W}_\nu \left[\underbrace{\mathbf{B}\mathbf{c} - \mathbf{p}^n}_{\text{change from the old state}} + \underbrace{\Delta t \nu \mathbf{\Lambda} \mathbf{B} \mathbf{c}}_{\text{diffusion}} + \underbrace{\Delta t \mathbf{q}(\mathbf{c})}_{\text{advection}} \right]. \quad (95)$$

Every quantity inside the brackets has $M = 32$ entries. Reading from left to right:

- $\mathbf{B}\mathbf{c}$ is the candidate new field projected onto the test functions.
- $\mathbf{p}^n$ is the already projected old field.
- Δt is the time-step size, here 0.005.
- ν is the viscosity for this trajectory.
- $\mathbf{\Lambda} \mathbf{B} \mathbf{c}$ is the projected negative-Laplacian contribution.
- $\mathbf{q}(\mathbf{c})$ is the quadratic tensor contraction from Equation (89).

The fixed diagonal weighting is

$$\mathbf{W}_\nu = \text{diag}\left(\frac{1}{1 + \Delta t \nu \lambda_1}, \dots, \frac{1}{1 + \Delta t \nu \lambda_M}\right). \quad (96)$$

It rescales high-frequency test equations, whose Laplacian eigenvalues are larger. It changes cheaply when the viscosity ν changes; the spatial bank $\mathbf{G}$, projected bank $\mathbf{B}$, eigenvalues, and tensor $\mathbf{Q}$ remain fixed.

For the Jacobian, the derivative of one quadratic entry is especially simple:

$$\frac{\partial q_m}{\partial \mathbf{c}} = (\mathbf{Q}_m \mathbf{c})^\top. \quad (97)$$

Stacking those rows into $\mathbf{D}_q(\mathbf{c}) \in \mathbb{R}^{M \times R}$ gives

$$\mathbf{J}_r = \mathbf{W}_\nu [\mathbf{B} + \Delta t \nu \mathbf{\Lambda} \mathbf{B} + \underbrace{\Delta t \mathbf{D}_q(\mathbf{c})}_{R \times K}] \frac{\partial \mathbf{h}}{\partial \mathbf{z}}. \quad (98)$$

The final Jacobian has shape $M \times K$: one row per test equation and one column per latent variable. In the code, automatic differentiation passes through the complete small reduced residual—the latent head plus its cached matrix and tensor contractions. It does not pass through a P -point advection stencil.

15 One complete Burgers rollout

For a new initial condition and viscosity, the online calculation is:

1. Receive the initial field $\mathbf{u}^0$ with P interior values.
2. Find an initial K -entry latent vector $\mathbf{z}^0$ whose decoded field $\mathbf{G}\mathbf{h}(\mathbf{z}^0)$ fits that initial field.
3. Compute its M -entry projection $\mathbf{p}^0 = \mathbf{B}\mathbf{h}(\mathbf{z}^0)$.
4. Extrapolate the last two latent states to obtain an initial guess for the next state. At the first step they are identical, so the initial code itself is used.
5. Repeatedly evaluate the head, the fixed matrix products, the quadratic tensor contraction, and Equation (95). Levenberg–Marquardt adjusts only the $K = 8$ latent variables.
6. Stop when the residual is sufficiently small, progress stalls, the proposed step is tiny, the damping reaches its maximum after rejected steps, the iteration budget is exhausted, or the initial residual is nonfinite. Carry the current latent state forward and record which stop condition occurred.
7. Set $\mathbf{p}^{n+1} = \mathbf{B}\mathbf{h}(\mathbf{z}^{n+1})$ and repeat until all 50 steps are complete.
8. Decode the saved latent states when the physical fields are needed.

The inner time-step solve uses the K -entry latent vector, the R -entry head output, the M -entry residual, one $M \times R$ matrix, and one $M \times R \times R$ tensor. It does not traverse the P physical-grid values. Fitting the initial condition and returning physical fields still touch the grid, so the claim of grid-independent cost applies to the cached latent solve, not to every part of the end-to-end workflow.

16 The important upwind sign caveat

The implemented full-order model uses sign-dependent upwinding:

$$(D^{\text{up}}u)_i = \begin{cases} (u_i - u_{i-1})/\Delta x, & u_i > 0, \\ (u_{i+1} - u_i)/\Delta x, & u_i \leq 0. \end{cases} \quad (99)$$

The advection at node i is $u_i(D^{\text{up}}u)_i$. A fixed backward-difference tensor is guaranteed to represent this expression exactly for a nonnegative decoded field. At $u_i = 0$, the forward/backward choice is irrelevant because the product is zero. Once negative values occur, the stencil changes and the tensor generally differs from the full sign-upwind operator, although special cancellations can still occur.

The same operator can be written as

$$\mathbf{N}_{\text{upwind}}(\mathbf{u}) = \mathbf{u} \odot D^c \mathbf{u} - \frac{\Delta x}{2} |\mathbf{u}| \odot \Delta_h \mathbf{u}. \quad (100)$$

The first term is quadratic and can be stored in a tensor. The second contains $|\mathbf{u}|$, which is not a polynomial in the state, so it still needs sampling or another approximation for genuinely sign-changing fields.

This distinction leads to three logically different Burgers models:

- **Sign-upwind plus a plain backward tensor:** guaranteed exact on the nonnegative part of state space; its general mismatch on sign-changing states must be measured, not hidden.

- **Sign-upwind in split form:** precompute the quadratic central term and sample only the $|u|$ stabilizer.
- **A skew-symmetric or centered quadratic discretization:** representable exactly by a tensor for either sign, but it is a different full-order discretization and must be judged against its own full-order solution.

For the current positive-blob family, the full-order fields are intended to remain nonnegative, while decoded candidates can undershoot slightly below zero. Therefore the plain tensor is algebraically exact on nonnegative decoded fields and only near the full sign-upwind oracle when those undershoots occur. It should not be described as universally exact for upwind Burgers.

17 Poisson and Burgers side by side

Question	Poisson	Burgers
Unknown online	one K -entry latent vector for a new source	one K -entry latent vector at each time step
Known problem data	projected source	viscosity and projected previous state
State degree	linear, degree 1	linear diffusion plus quadratic advection
Main cached object	$M \times R$ matrix	$M \times R$ matrix and $M \times R \times R$ tensor
Full grid in the inner solve?	no	no for the tensor arm
Main limitation	new operator or boundary data require rebuilding	non-polynomial sign-upwind part is not one global tensor

Answer to “Is a fixed matrix generalizable?” Yes, within the problem family for which it was built. The fixed matrix is not a stored answer; it is a compact representation of how the fixed PDE operator acts on the learned spatial patterns. New data change the latent coefficients and therefore change the solution. If the operator has affine parameter dependence, one matrix can be stored for each fixed component and combined online. A new PDE, boundary condition, non-affine operator, or unvalidated state family requires new projected objects and possibly new training.

18 The full NM-ROM pipeline in fifteen lines

1. Generate full-order solution snapshots from the intended PDE family.
2. Give each selected snapshot a trainable K -entry latent code.
3. Train the spatial network, latent head, and codes to reconstruct the snapshots.
4. Freeze both networks.
5. Evaluate the spatial network to build the $P \times R$ bank $\mathbf{G}$.
6. Choose M smooth test functions and build Φ .

7. Precompute an $M \times R$ matrix for every linear state term.
8. Precompute an $M \times R \times R$ tensor for every quadratic state term.
9. Identify every non-polynomial term that still requires sampling.
10. Receive new problem data: a source for Poisson, or an initial condition and viscosity for Burgers.
11. Obtain an initial K -entry latent guess.
12. Evaluate only the small head and cached reduced operators inside the latent solve.
13. Use the $M \times K$ residual Jacobian to update the latent vector.
14. For an unsteady PDE, carry the accepted latent state into the next time step.
15. Decode $\mathbf{Gh}(\mathbf{z})$ when the P -value physical field is needed.

Plain-language glossary

K — **latent variables**. The number of entries in $\mathbf{z}$ that the online solver adjusts.

R — **learned spatial patterns**. The number of columns in $\mathbf{G}$ and, equivalently, the number of coefficients returned by $\mathbf{h}(\mathbf{z})$.

M — **test equations**. The number of weak test equations retained by the ROM.

P — **physical-grid values**. The number of interior grid values stored in one solution vector $\mathbf{u}$.

$\mathbf{g}(\mathbf{x})$ — **spatial neural network**. Returns the raw learned-pattern values at coordinate $\mathbf{x}$.

$\mathbf{h}(\mathbf{z})$ — **latent neural network**. Returns the coefficients that combine the learned spatial patterns.

$\mathbf{G}$ — **frozen spatial bank**. Contains the learned patterns evaluated at every interior grid point, including the fixed output scale and hard boundary mask.

Φ — **test-function matrix**. Its columns are the selected sine test functions.

$\mathbf{B}$ — **projected bank**. This is $\Phi^\top \mathbf{G}$.

Λ — **eigenvalue matrix**. The diagonal matrix of Laplacian eigenvalues for the sine test modes.

$\mathbf{A}$ — **projected Poisson operator**. This equals $\Lambda \mathbf{B}$ in this setting.

$\mathbf{W}$ — **residual weighting**. The fixed inverse-eigenvalue scaling.

$\mathbf{f}_M$ — **projected forcing**. The new forcing projected onto the retained test functions.

$\mathbf{r}(\mathbf{z})$ — **residual**. The weighted mismatch between the decoded field's projected Poisson response and the projected forcing, or between two consecutive states and the projected time-dependent PDE terms for Burgers.

$\mathbf{T}$ — **raw Burgers interaction tensor**. Stores the projection of every ordered pair consisting of one learned pattern and one backward-differenced learned pattern.

$\mathbf{Q}$ — **symmetric Burgers interaction tensor**. The raw tensor plus its transpose in the two learned-pattern indices. It is used in the quadratic contraction and its derivative.

$\mathbf{q}(\mathbf{h})$ — **projected advection**. The M -entry vector produced by contracting the Burgers tensor with the R -entry coefficient vector twice.

Δt — **time-step size**. The amount of simulated time advanced by one backward-Euler solve.

ν — **viscosity**. The Burgers parameter controlling the strength of diffusion.

FOM — **full-order model**. The reference discretized PDE solver that evolves all physical-grid values.

LM — **Levenberg–Marquardt**. The damped nonlinear least-squares method used to adjust the latent variables.

NM-ROM — **nonlinear-manifold reduced-order model**. A reduced solver whose admissible states are selected by a nonlinear map from the latent vector, here $\mathbf{u} = \mathbf{G}\mathbf{h}(\mathbf{z})$.

Polynomial degree in the state. The highest number of state factors multiplied together in a residual term: one for a linear term, two for a quadratic term.

Sign-upwind. An advection difference that switches between a backward and forward stencil according to the sign of the local velocity.

Tensor contraction. Summing the stored pairwise interaction coefficients against every product $h_j h_k$ to produce the projected quadratic term.

Auto-decoder. A decoder trained with one free latent code per training snapshot, without a learned encoder.

Offline. Training and precomputation performed before solving new source instances.

Online. The latent solve and final field reconstruction for a new source.